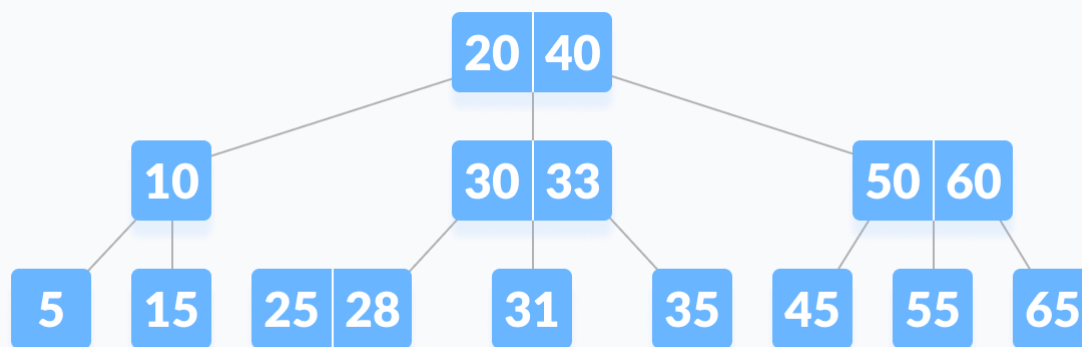


B-tree

In search trees like binary search tree, AVL Tree, Red-Black tree, etc., every node contains only one value (key) and a maximum of two children. But there is a special type of search tree called B-Tree in which a node contains more than one value (key) and more than two children. B-Tree was developed in the year 1972 by Bayer and McCreight with the name Height Balanced m-way Search Tree. Later it was named as B-Tree.

B-tree is a special type of self-balancing search tree in which each node can contain more than one key and can have more than two children. It is a generalized form of the binary search tree.

It is also known as a height-balanced m-way tree.



B-tree of order 3

Why do you need a B-tree data structure?

The need for B-tree arose with the rise in the need for lesser time in accessing the physical storage media like a hard disk. The secondary storage devices are slower with a larger capacity. There was a need for such types of data structures that minimize the disk accesses.

Other data structures such as a binary search tree, avl tree, red-black tree, etc can store only one key in one node. If you have to store a large number of keys, then the height of such trees becomes very large and the access time increases.

However, B-tree can store many keys in a single node and can have multiple child nodes. This decreases the height significantly allowing faster disk accesses.

B Tree Applications:

- Database and File Systems
- To store blocks of data in secondary storage media
- Multilevel Indexing

B-tree Properties

B-Tree of Order m has the following properties...

Property #1 - All leaf nodes must be at **same level**.

Property #2 - All nodes except root must have at **least $\lceil m/2 \rceil - 1$ keys** and **maximum of $(m-1)$ keys**.

Property #3 - All non-leaf nodes except root (i.e. **all internal nodes**) must have at **least $\lceil m/2 \rceil$ children**.

Property #4 - If the root node is a non-leaf node, then it must have at least 2 children.

Property #5 - A non-leaf node with $n-1$ keys must have n number of children.

Property #6 - All the key values in a node must be in **Ascending Order**.

Order of BTree (m)	MAXIMUM	MINIMUM
Children	m	$\lceil m/2 \rceil$
Keys	$m-1$	$\lceil m/2 \rceil - 1$

Operations on a B-tree

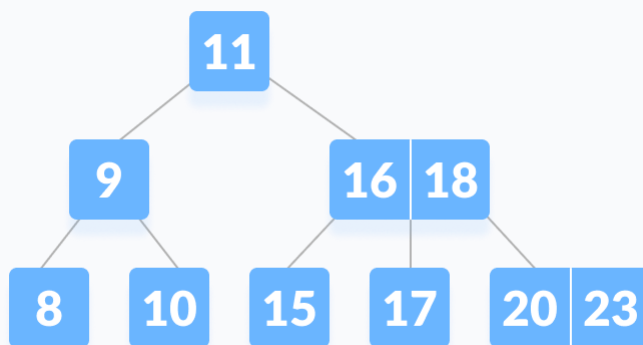
Searching an element in a B-tree

Searching for an element in a B-tree is the generalized form of searching an element in a Binary Search Tree. The following steps are followed.

1. Starting from the root node, compare k with the first key of the node.
If $k = \text{the first key of the node}$, return the node and the index.
2. If $k.\text{leaf} = \text{true}$, return `NULL` (i.e. not found).
3. If $k < \text{the first key of the root node}$, search the left child of this key recursively.
4. If there is more than one key in the current node and $k > \text{the first key}$, compare k with the next key in the node.
If $k < \text{next key}$, search the left child of this key (ie. k lies in between the first and the second keys).
Else, search the right child of the key.
5. Repeat steps 1 to 4 until the leaf is reached.

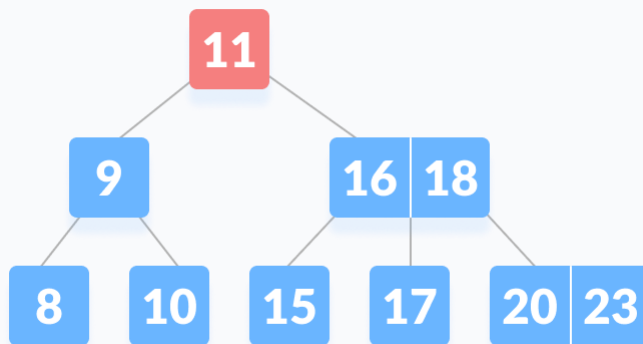
Searching Example

1. Let us search key $k = 17$ in the tree below of degree 3.



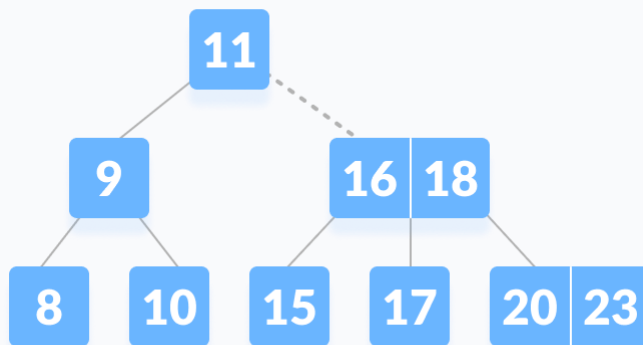
B-tree

2. k is not found in the root so, compare it with the root key.



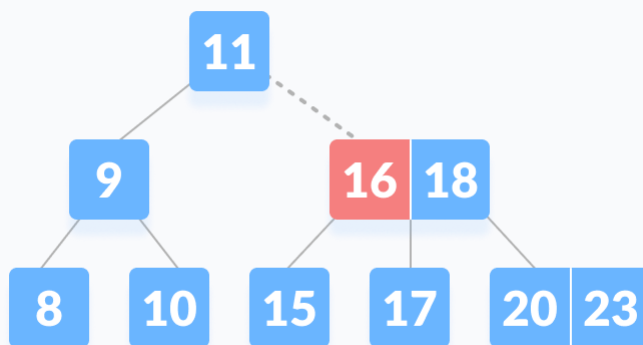
k is not found on the root node

3. Since $k > 11$, go to the right child of the root node.



Go to the right subtree

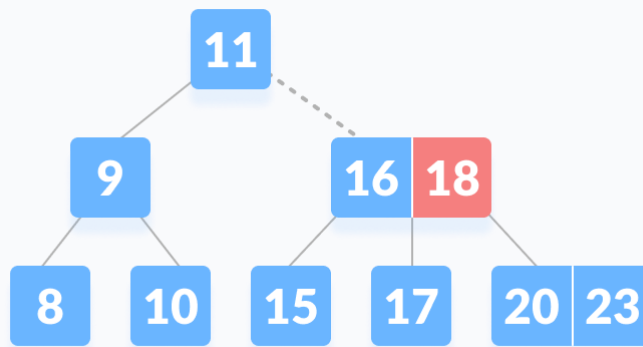
4. Compare k with 16. Since $k > 16$, compare k with the next key 18.



Compare with the keys from left to

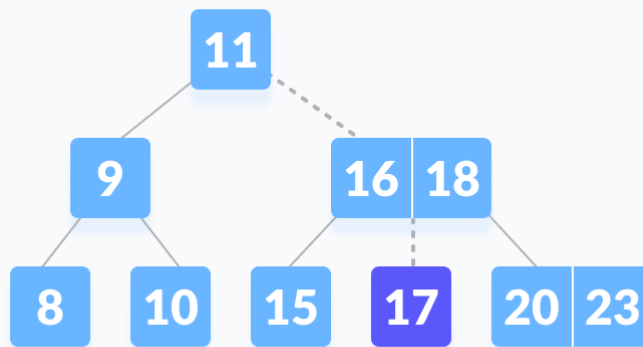
right

5. Since $k < 18$, k lies between 16 and 18. Search in the right child of 16 or the left



child of 18.

6. k lies in between 16 and 18



7. k is found.

k is found

Insertion into a B-tree

Inserting an element on a B-tree consists of two events: **searching the appropriate node** to insert the element and **splitting the node** if required.

Insertion operation always takes place in the bottom-up approach.

Let us understand these events below.

Insertion Operation

In a B tree, all insertions are done at the leaf node level. A new value is inserted in the B tree using the algorithm given below.

1. Search the B tree to find the leaf node where the new key value should be inserted.

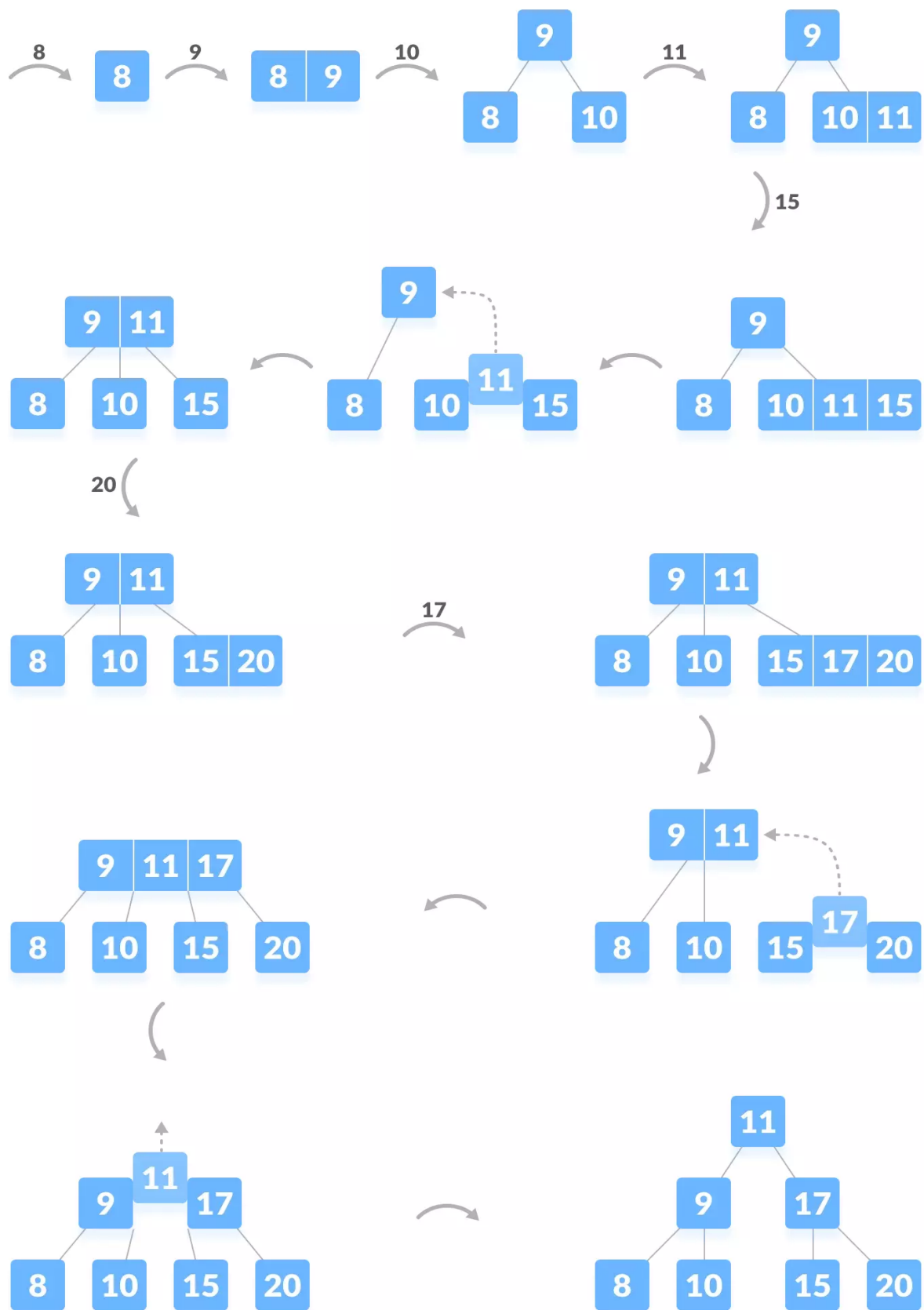
2. If the leaf node is not full, that is, it contains less than $m-1$ key values, then insert the new element in the node keeping the node's elements ordered.
 3. If the leaf node is full, that is, the leaf node already contains $m-1$ key values, then:
 - (a) insert the new value in order into the existing set of keys,
 - (b) split the node at its median into two nodes (note that the split nodes are half full), and
 - (c) push the median element up to its parent's node. If the parent's node is already full, then split the parent node by following the same steps.
-

Insertion Example

Let us understand the insertion operation with the illustrations below.

The elements to be inserted are 8, 9, 10, 11, 15, 20, 17.

Inserting elements into a B-tree of order 3.



The **degree of a tree** generally refers to the **number of children** or **subtrees** a node has. In the context of specific types of trees, the definition of **degree** can vary slightly.

General Definition: Degree of a Tree

- In a general tree, the **degree of a node** is the number of children that node has.
- The **degree of a tree** is the degree of the node with the most children (i.e., the maximum degree of any node in the tree).

For example:

- If a node has 3 children, its degree is 3.
- If another node has 2 children, its degree is 2.
- The degree of the whole tree is the maximum degree among all nodes.

Degree in the Context of a B-tree

In the context of a **B-tree**, the term **degree** usually refers to the **minimum degree**, denoted as **t**.

This is a critical parameter that controls the structure of the B-tree, dictating how many keys and children each node can have.

It directly impacts the height and balance of the B-tree. A higher degree allows each node to hold more keys and children, making the tree **shorter and wider**, which optimizes operations like search, insert, and delete.

Minimum Degree (t) of a B-tree

- The **minimum degree t** in a B-tree is a constant value that governs the **minimum and maximum number of keys** a node can contain and the **minimum and maximum number of children** a node can have.

Degree in Terms of B-tree Structure:

1. Number of Keys per Node:

- A node (except the root) must contain **at least $t - 1$ keys**.
- A node can contain at most **$2t - 1$ keys**.

2. Number of Children per Node:

- A node (except a leaf) must have **at least t children** if it contains the minimum number of keys.
- A node can have **at most $2t$ children** if it contains the maximum number of keys.

Minimum Degree of BTree (t)	MAXIMUM	MINIMUM
Children	$2t$	t
Keys	$2t-1$	$t - 1$

Explanation with an Example:

Consider a B-tree with **minimum degree $t = 3$** :

- **Minimum keys per node:** Each node (except the root) must have at least $t - 1 = 2$ keys.
- **Maximum keys per node:** Each node can have up to $2t - 1 = 5$ keys.
- **Minimum children per node:** Each internal node (except the root) must have at least $t = 3$ children.
- **Maximum children per node:** Each internal node can have at most $2t = 6$ children.

Root Node Special Case:

- The **root node** of a B-tree is allowed to have fewer than $t - 1$ keys. If the root is the only node, it can have between **1** and $2t - 1$ keys.

Degree in Relation to Tree Balance:

The **degree (t)** controls how wide the B-tree spreads (how many children nodes can have), which in turn determines its height. A higher **degree** means that nodes can hold more keys, making the tree **wider** but **shorter**. This reduces the number of levels the tree must have, which can make searching, inserting, and deleting faster because fewer levels need to be traversed.

Example of a B-tree with $t = 3$:

Let's walk through an example with $t = 3$:

1. Every non-root node must have **at least 2 keys** and **at least 3 children**.
2. A node can have **at most 5 keys** and **at most 6 children**.
3. Suppose a node contains 4 keys. This node would have between **5 to 6 children**.
4. If a node has **only 2 keys**, it must have exactly **3 children**.

Thus, **degree** helps control the balance between the width and height of the B-tree, ensuring that it remains efficient for operations like search, insert, and delete by keeping the tree balanced and shallow.

Summary:

- The **degree of a B-tree** refers to its **minimum degree (t)**, which controls the **number of keys and children** a node can have.
-

BCA 2022-23

1. How B-Tree differs with Binary Tree structures ?

Show insertion of the following keys into an empty B-tree.

Keys : 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110
and 120 of minimum degree 't' as 3.

Deletion from a B-tree

Deleting an element on a B-tree consists of three main events: **searching the node where the key to be deleted exists**, deleting the key and balancing the tree if required.

While deleting a tree, a condition called **underflow** may occur. Underflow occurs when a node contains less than the minimum number of keys it should hold.

The terms to be understood before studying deletion operation are:

1. Inorder Predecessor

The largest key on the left child of a node is called its inorder predecessor.

2. Inorder Successor

The smallest key on the right child of a node is called its inorder successor.

Deletion Operation

Before going through the steps below, one must know these facts about a B tree of degree or order **m (here m=3)**.

1. A node can have a maximum of m children. (i.e. 3)
2. A node can contain a maximum of $m - 1$ keys. (i.e. 2)
3. A node should have a minimum of $\lceil m/2 \rceil$ children. (i.e. 2).ceil(m/2)
4. A node (except root node) should contain a minimum of $\lceil m/2 \rceil - 1$ **keys**. (i.e. 1)

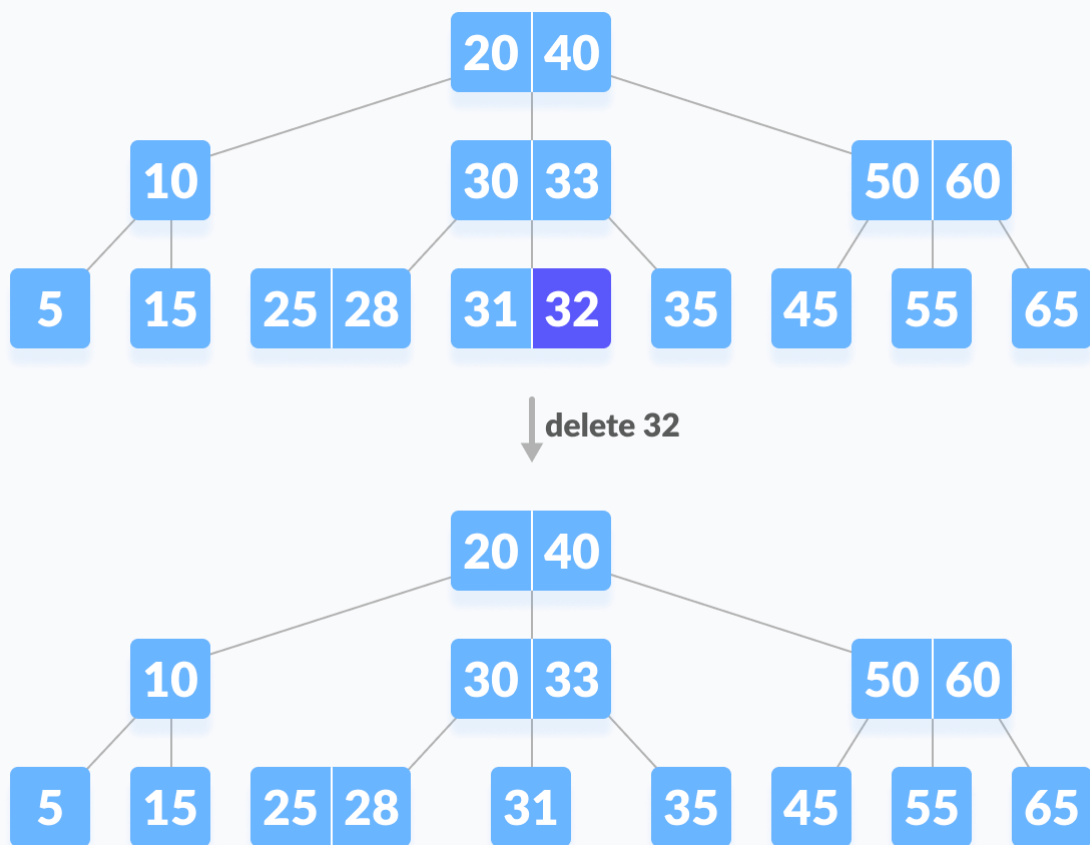
There are three main cases for deletion operation in a B tree.

Case I

The key to be deleted lies in the leaf. There are two cases for it.

1. The deletion of the key does not violate the property of the minimum number of keys a node should hold.

In the tree below, deleting 32 does not violate the above properties.



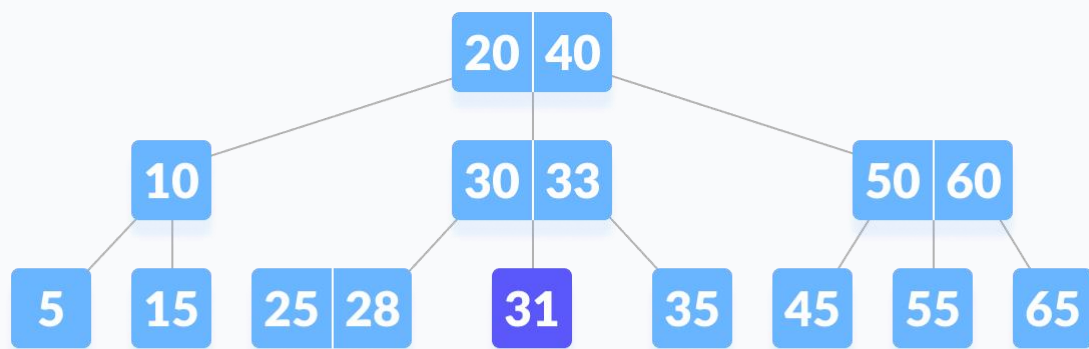
Deleting a leaf key (32) from B-tree

2. The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighboring sibling node in the order of left to right.

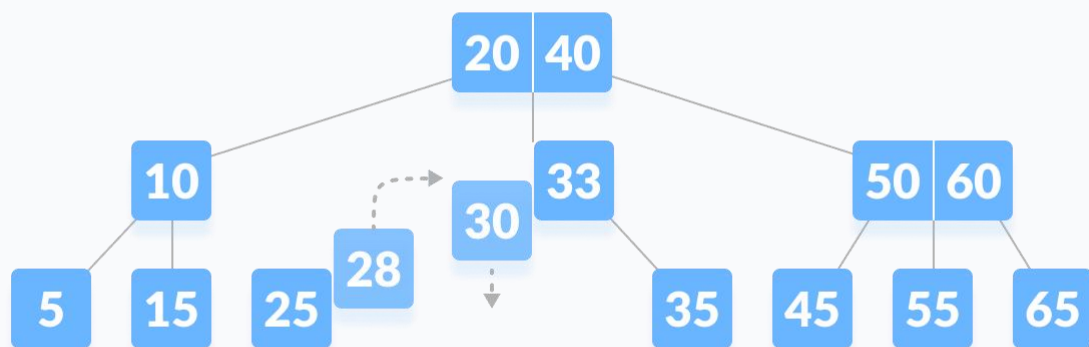
First, visit the immediate left sibling. If the left sibling node has more than a minimum number of keys, then borrow a key from this node through parent.

Else, check to borrow from the immediate right sibling node through parent.

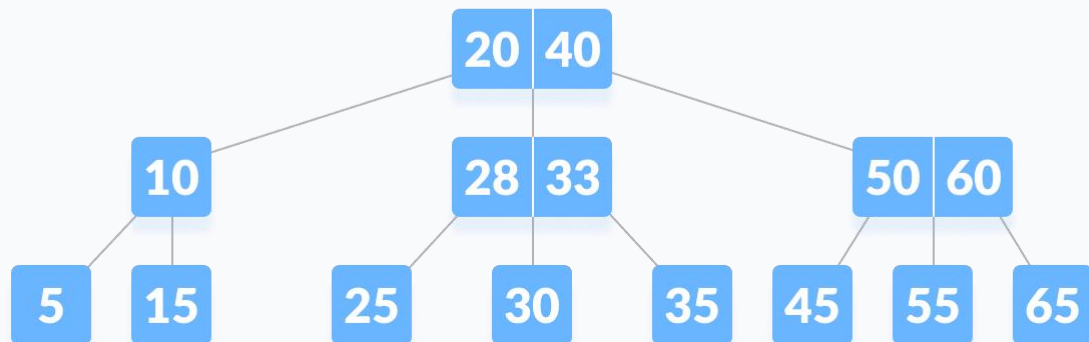
In the tree below, deleting 31 results in the above condition. Let us borrow a key from the left sibling node.



↓ delete 31



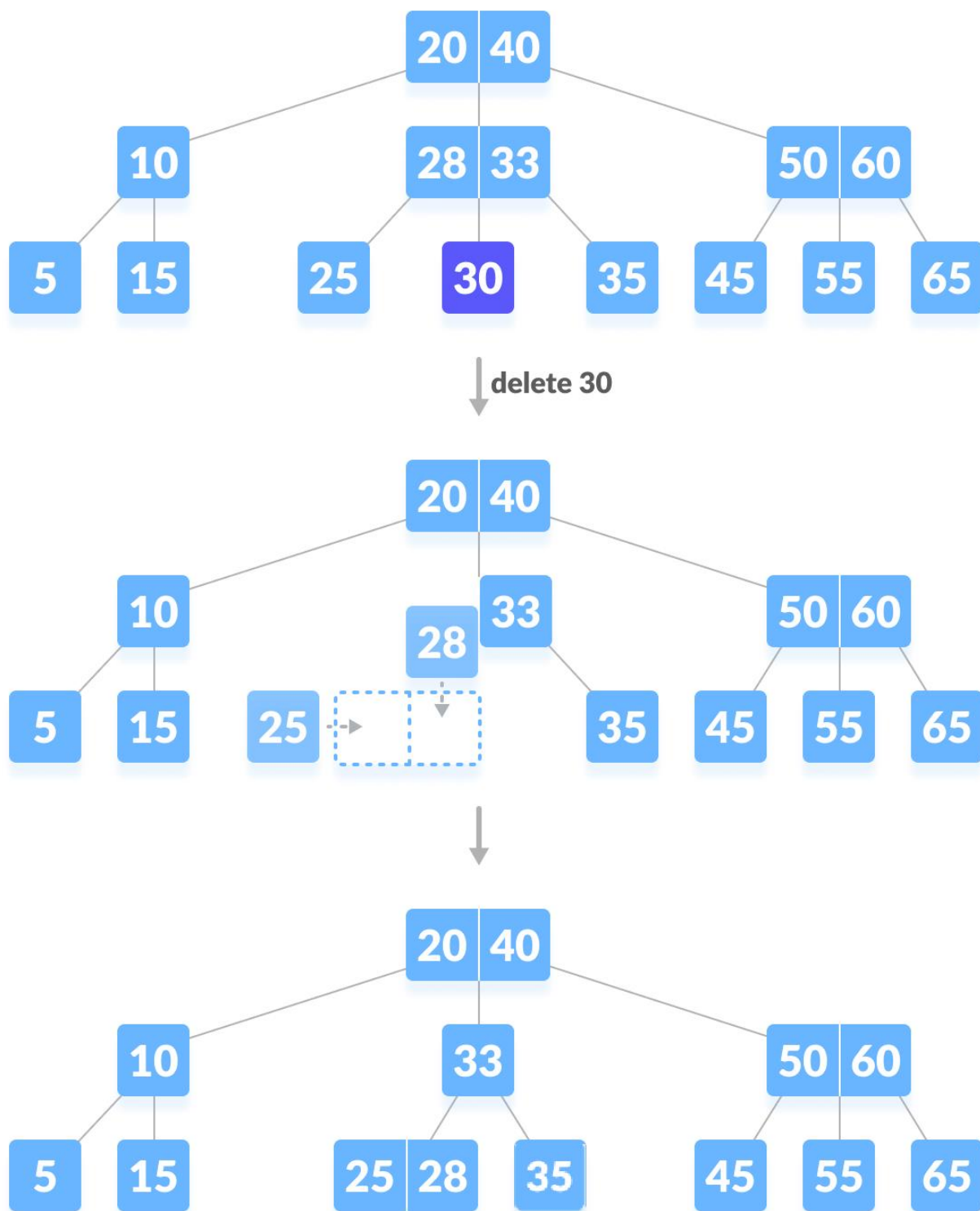
↓



Deleting a leaf key (31)

If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**

Deleting 30 results in the above case.

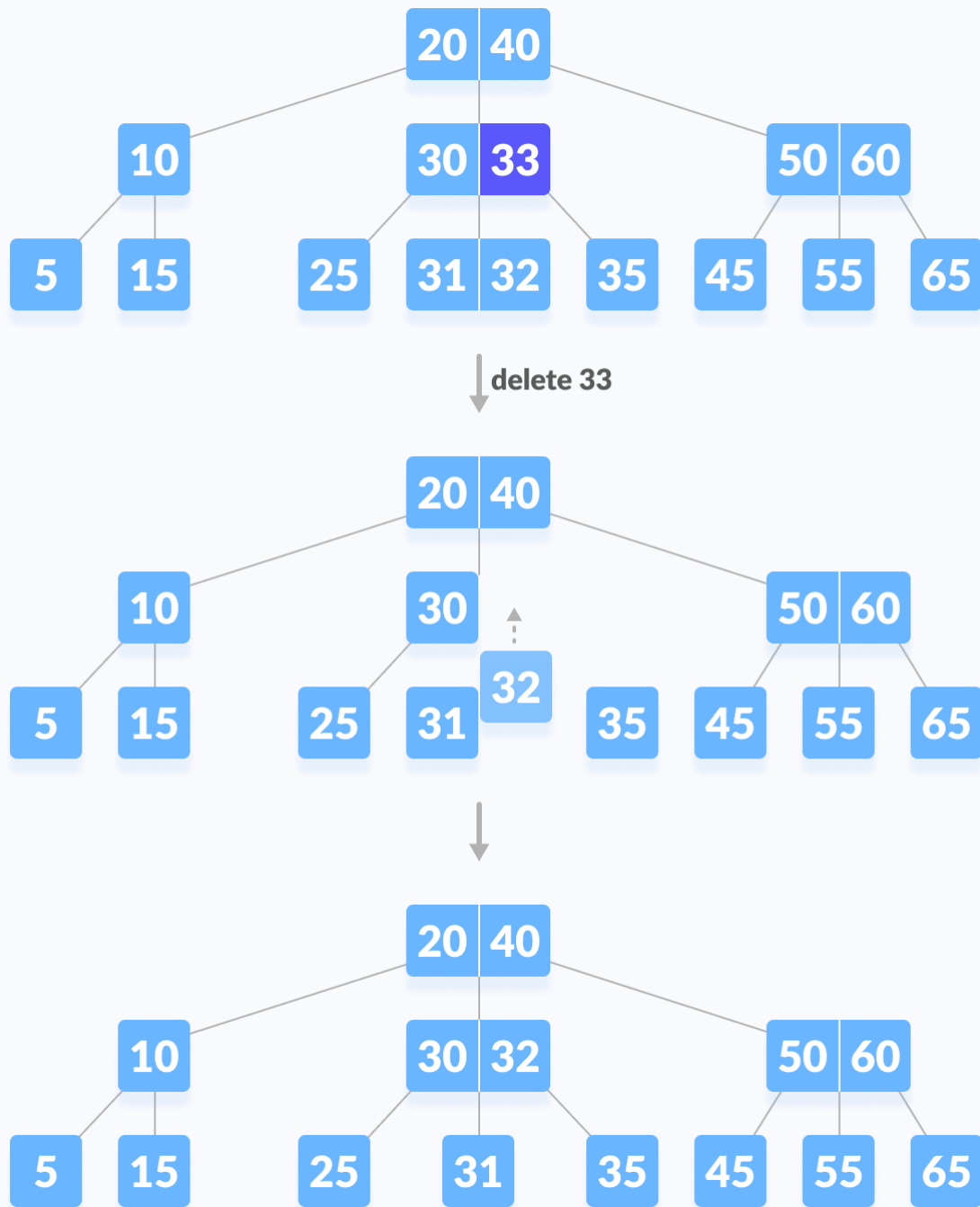


Delete a leaf key (30)

Case II

If the key to be deleted lies in the internal node, the following cases occur.

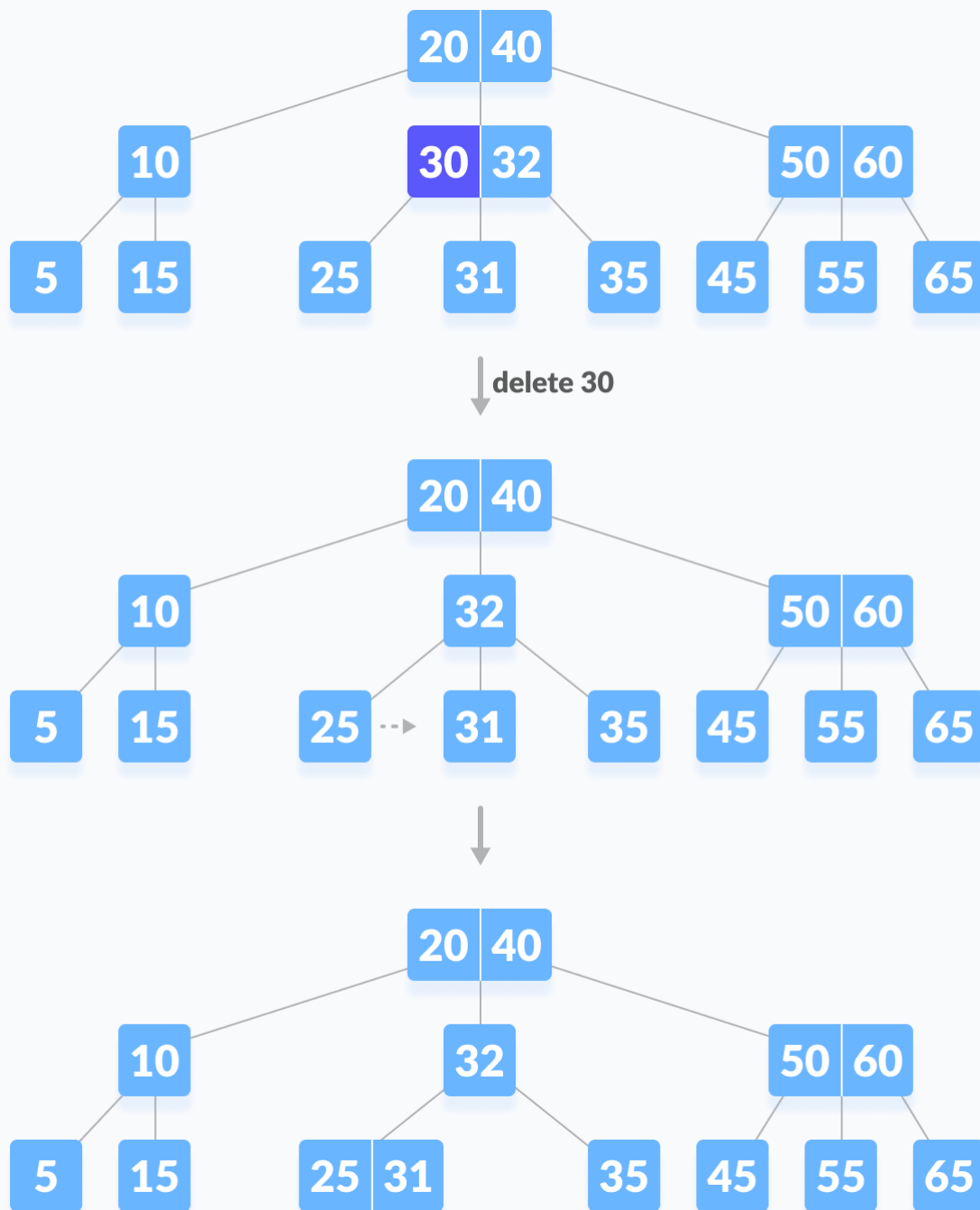
1. The internal node, which is deleted, is replaced by an inorder predecessor if the left child has more than the minimum number of keys.



Deleting an internal node (33)

2. The internal node, which is deleted, is replaced by an inorder successor if the right child has more than the minimum number of keys.

3. If either child has exactly a minimum number of keys then, merge the left and the right children.



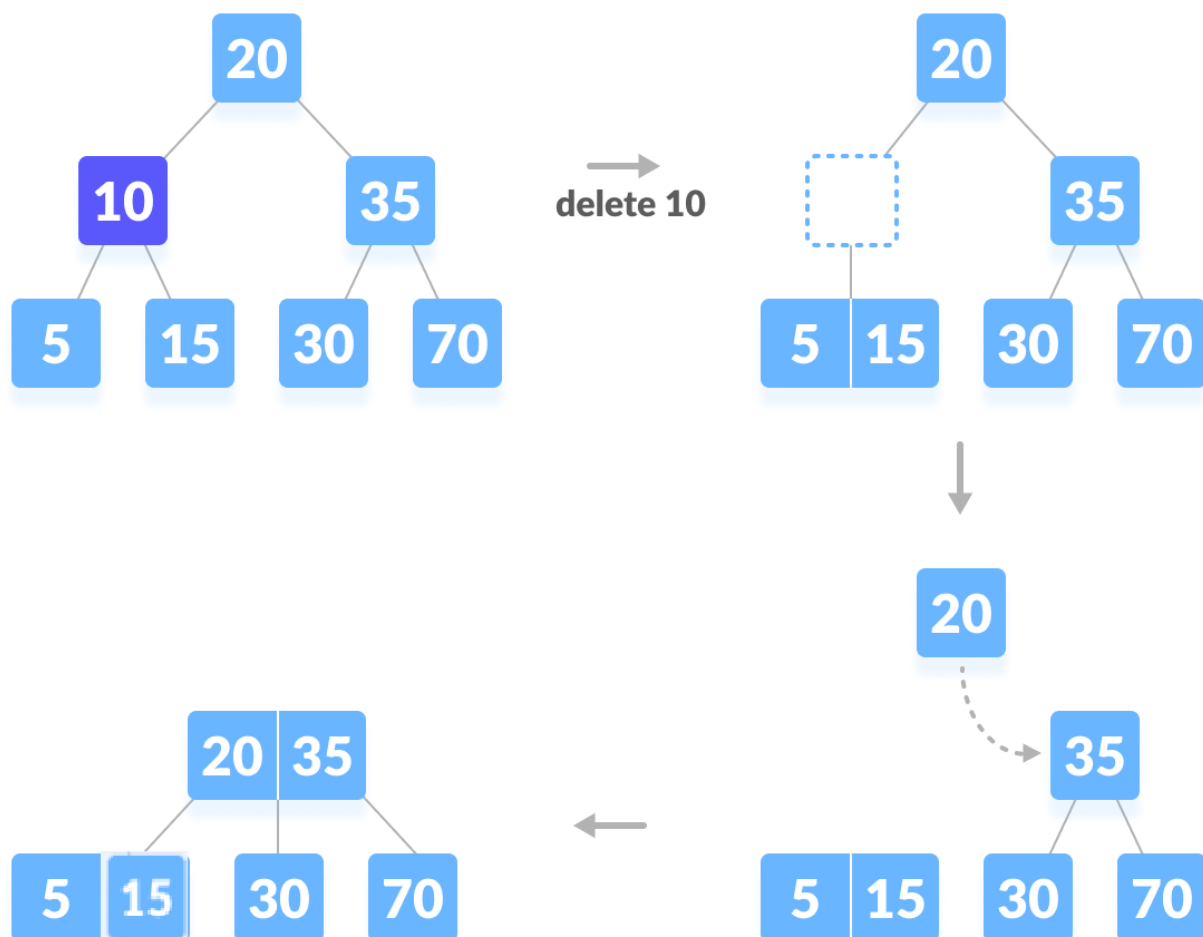
Deleting an internal node (30).

After merging if the parent node has less than the minimum number of keys then, look for the siblings as in Case I.

Case III

In this case, the height of the tree shrinks. If the target key lies in an internal node, and the deletion of the key leads to a fewer number of keys in the node (i.e. less than the minimum required), then look for the inorder predecessor and the inorder successor. If both the children contain a minimum number of keys then, borrowing cannot take place. This leads to Case II(3) i.e. merging the children.

Again, look for the sibling to borrow a key. But, if the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).



Deleting an internal node (10)